Entrega 2 technical report

The implementation translates all valid AST construction kinds into typed, source-located TAC through one CLI/IDE core, with static extended-symbol layouts. T9/T10 resolved the initial accepted-source gaps, including print, remainder, lazy ternaries, assignment values, undefined/null storage, untyped values and try/catch. This report maps the four-page assignment to implemented contracts and executable evidence, not exhaustive program correctness, a future grade or browser certification.

Basis and outcome

Normative reference: canonical instructions, a complete Spanish transcription of pages 1–4 of the human-provided `Proyecto_2__Compiladores.pdf`, including page boundaries and the rubric. The original remains local, ignored and unchanged; its SHA-256 is `0ba21279ad758dd8ef69344adae9d31193d5ca4d064ce525257da6974c707bdc`. A fresh Git checkout includes the transcription and does not require the original PDF. The matrix below retains short Spanish quotations; page numbers refer to original PDF pages, not repository line numbers.

The key result is one compiler with explicit ownership, guarded runtime operations and atomic rejection of actual compilation diagnostics. Architecture explains the decisions; TAC design specifies the implemented IR, examples and assumptions. Run instructions and acceptance provide the executable entry points. This final refresh follows published T9 `9ce55ce` and T10 `633aad1`, including independent verification and the T10 field-contract correction. No new concrete normative implementation gap is established by this refresh; the remaining human/evaluation checks are listed below.

PDF requirement and evidence matrix

Compiler files below are under `internal/compiscript`; lowering filenames are in its `tac` subdirectory. Paths starting with `cmd`, `testdata` or `web` are repository-root relative. Tests are concrete evidence entry points, not a claim of measured line coverage. “Implemented” means the named capability has implementation and executable tests; it is not a guarantee for every possible source program.

Page / short verbatim requirement	Implementation and evidence	Outcome
1: “traducen cada construcción de Compiscript”	T9/T10 core/built-CLI suites and retained `independent_t9_verifier_test.go`, `independent_t10_verifier_test.go`, `independent_t10_b1_recheck_test.go`; AST kind inventory	All 18 valid statement / 14 expression kinds dispatched; initial gap resolved, explicit static/runtime guards retained
1: “asignación, operaciones binarias y unarias, copia”	`tac/ir.go`, `expression.go`, `values_effects_test.go`, `dynamic_exceptions_test.go`; numeric translation in reference	Implemented typed/guarded operations, remainder and value-producing assignments
1: “saltos condicionales e incondicionales, etiquetas”	`tac/flow.go`, `loops.go`, `switch.go`; `conditions_test.go`, `loops_test.go`, `switch_test.go`; checked-in examples	Implemented

Page / short verbatim requirement	Implementation and evidence	Outcome
1: “llamadas a procedimientos, acceso indexado”	`tac/calls.go`, `collections.go`; `functions_test.go`, `collections_test.go`	Implemented
1: “consistente y estar documentada”	Instruction reference, typed text/JSON, formatting tests `tac/ir_test.go`, `procedures_test.go`	Documented implemented syntax
1: “(+, -, *, /) respetando precedencia y asociatividad”	`tac/expression.go`, `TestCompileNumeric`; `numeric.cps`, explicit `i2f` and float division	Implemented named arithmetic
1: “(&&, ||, !) y de comparación (==, !=, <, <=, >, >=)”	`tac/flow.go`, `conditions_test.go`; short-circuit execution/trap checks	Implemented named logic/comparisons
1: “declaraciones y asignaciones de variables y constantes”	`tac/lower.go`; `compile_test.go`, semantic const tests, `TestCollectionAtomicRejection`, values/effects and dynamic/exception suites	Typed and unknown undefined declarations; no fabricated default; runtime guards preserve const/type contracts
2: “if/else, while, do-while, for y switch”	`flow.go`, `loops.go`, `switch.go`; named example files under `testdata/compiscript/tac`, `TestAcceptanceValidLoops`	Implemented within current semantic contracts (boolean switch)
2: “break y continue”	Shared transfer stack; loop/switch/collection tests, mixed `acceptance.cps`	Implemented nearest eligible targets
2: “paso de parámetros, llamadas y valores de retorno”	`tac/functions.go`, `calls.go`, `frames.go`, `dynamic.go`; frame and dynamic/exception suites	Typed/untyped formals with actual runtime guards; missing result annotation remains null
2: “funciones recursivas”	Fixed-point dependencies and fresh frames; `recursion_test.go`, `recursion_dependencies_test.go`, `recursion.cps`	Implemented self/mutual recursion; termination not promised
2: “funciones anidadas y closures”	`tac/closures.go`; `TestClosureExecution`, `closure_compositions_test.go`, `closures.cps`	Implemented retained shared cells and escaped aliases
2: “creación de objetos e invocación del constructor”	`tac/objects.go`; `TestObjectExecution`, `TestObjectThreeLevelPolicyA`, `objects.cps`	Implemented explicit initialization policy A
2: “acceso a atributos” / “incluyendo el uso de this”	Field loads/stores and bound methods; `objects_test.go`, `independent_r3_test.go`, T10 field-contract regressions	Nominal/inherited receivers and checked dynamic access; unannotated fields remain unknown, annotated fields stay constrained
2: “creación de listas” / “mediante índices”	`tac/collections.go`; `TestCollectionExecution`, retained T3/T9/T10 tests, `collections.cps`	Empty/nested/ragged/null lists, reads/stores and checked unknown views; foreach additionally supported
2: “Algoritmo de asignación de temporales”	`tac/temps.go`; `tac/temps_test.go`, `TestCompileRepeatedExpressions`	Smallest free slot, independent pools per tape
2: “Reciclaje de temporales una vez que su valor ha sido consumido”	Final-consumer release; live-range/reuse tests in `tac/temps_test.go`; lifetime contract	Implemented, no optimality claim

Page / short verbatim requirement	Implementation and evidence	Outcome
2: “direcciones de memoria o desplazamientos, tamaños, etiquetas”	`tac/symbols.go`; `TestCompilationSymbols`, `tac/symbols_test.go`, `symbols_cli_test.go`	Deterministic abstract-cell offsets/sizes and actual labels, not physical addresses
2: “parámetros, variables locales, temporales, valor y dirección de retorno, enlaces de control y de acceso”	`tac/frames.go`, retained environment layouts; `TestFunctionFrameMetadata`, `TestCompilationSymbols`	Explicit frame control/result cells and lexical links; layout reference
2: “interactuar con cada fase de la compilación”	AST/source spans -> semantic identities/ scopes -> Generate -> codegen locations; `TestAcceptanceMixedExecution` immutability checks	Same identity inventory, separate non-mutating compilation report
2: “extensión .cps” / “Código intermedio (TAC)” / “Reporte de errores” / “Estado de la tabla de símbolos”	CLI extension/UTF-8 checks; report `tac`, `diagnostics`, `scopes`, optional `codegen`; `cmd/compiscript-tac/main_test.go`, CLI symbol tests	Implemented text/JSON and IDE views
3 step 1: “carga o escribe un archivo .cps a través del IDE”	`web` load/edit/save and compile controls; Node upload and stale-request tests; IDE guide	Wiring tested; manual browser check pending
3 steps 2–4: “(ANTLR) construye el árbol sintáctico” / “validando las reglas” / “Si no existen errores”	`frontend.Parse` -> `semantic.Analyze` -> `tac.Generate` in Compile; facade parity and atomic recovery tests	Implemented once per compilation; no second compiler
3 step 5: “errores encontrados con su tipo y ubicación”	Typed phase/code/spans; CLI rendering and `tacText` on HTTP; `TestMixedAcceptanceAPI`, Node diagnostic rendering tests	Implemented; HTTP 200 language errors differ from transport 4xx
3 step 6 and deliverable: “batería de pruebas” / “casos exitosos y fallidos”	Runnable battery, public CLI/API tests and bounded oracle execution	Passed focused checks; evaluation-day availability still human responsibility
3: “repositorio de GitHub, con commits individuales”	Root repository link and audited commit history below	Local author records established; real contribution attestation and remote/ evaluation access pending
3: “Documentación de la arquitectura” / “cómo ejecutar el compilador”	Architecture, run guide and root index	Versioned Markdown supplied
3: “lenguaje intermedio diseñado, con ejemplos de traducción y los supuestos”	TAC reference: numeric translation, frames, closures, objects/lists, values/effects, guarded values and handlers	Supplied with explicit initialization, formatting, fault and unwinding assumptions
3: “IDE funcional que permita escribir y compilar”	Shared core HTTP endpoint plus TAC/errors/ symbol panels; Go httptest and 39 Node cases	Functional boundary tested; real visual evaluation pending

Page / short verbatim requirement	Implementation and evidence	Outcome
4: “funcionar correctamente el día de la presentación” / “documentado y organizado”	Documentation index and reproducible tests; manual preparation checklist below	Cannot certify future presentation conditions
4: “Diseño de CI” 25 / “Generación de TAC” 65 / “Tabla de Símbolos” 10	Reference/design, executable generation battery, extended-symbol/layout report respectively	Grading weights totaling 100 points, **not completion percentages**

Observed verification and provenance

The parent-provided implementation records distinguish each unit's proof from later test-only additions. T9 and final T10 each report isolated full Go suites (18 tested packages, four without tests), build, forced vet and 39 Node cases passing. T10's final nominal-field **test-only** addition followed its full-suite proof; the parent reran all four affected packages afterward, with production unchanged. This documentation-only refresh did not rerun full generation/build/vet and does not claim a fresh full-suite result after `633aad1`.

Historical evidence includes exact compatibility comparisons: T4 writer 45, independent verifier 60 plus nine panic-to-atomic changes; T6 39 comparisons stripping only the authorized additive `codegen` field and 36 repeated JSON checks; T7 writer 37 HTTP, seven DOM and 31 CLI comparisons, with independent API/UI checks. These are overlapping scoped sets, **not an additive grand total**. Original T7 RED evidence is a retained transcript (API 404; Node 14 failures/four passes), not a separately saved raw log artifact. Historical native approval applies to T1 only; later clone-local review was disabled/unmanaged, not approved.

Published unit	Parent-recorded implementation and independent evidence
T9 `9ce55ce`	31 shared core/CLI scenarios and 82 exact baseline comparisons; independent 20 execution/output cases, 30 old-fixture comparisons, 20 Analyze comparisons, ten atomic rejections and four CLI-policy checks. Durable verifier retained and four packages rerun by parent
T10 `633aad1`	69 shared core/CLI scenarios plus fatal checks; final 170 exact comparisons (33 old-fixture text/JSON/Analyze, 110 new Analyze, 27 rejected reports) and 51 source overlays. Independent broader check: 29 passing compositions and 83 exact comparisons before B1; five unknown-field failures corrected once, then all five plus eight direct regressions passed in a limited recheck

The T10 B1 correction supersedes T9's private null-field narrowing: no annotation means unknown field storage even when initialized to null/numeric values, while missing initializers remain undefined. Explicit annotated nominal fields retain their guards before effects. Both independent T10 files are durable current-core/current-built-CLI tests, not tests requiring external baseline controls. The overlapping scenario/comparison counts above must not be summed into a coverage total.

The initial T8 audit at `f416fa2` correctly observed eight Analyze-valid sources rejected during lowering, including `let x=1; print(1);` (`TAC_UNSUPPORT`

ED at 1:10). That is historical RED evidence for the requirement gap, **not current behavior**. Its original documentation checks covered ten fixtures/20 CLI invocations; the final refresh below covers twelve/24.

Fresh final-refresh verification used offline Go with external HOME/TMPDIR/GOCACHE and an existing module cache:

- Built TAC and Analyze binaries outside the repository; help, all twelve checked-in TAC examples in text/JSON, rejected text/JSON, invalid-format status and legacy Analyze statuses were checked. All 24 successful example invocations exited 0 with empty stderr; source hashes were unchanged. No product execution service was introduced or invoked.
- `go test ./internal/compiscript ./cmd/ide -run '^Test(ValuesEffects|DynamicExceptions|IndependentT9|IndependentT10|MixedAcceptanceAPI)' -count=1`: both packages passed, including retained independent regressions, values/output/fault effects, handler fatal boundaries and HTTP parity.
- The mixed oracle expectations remain `first=30`, `second=43`, `independent=7`, `continued=57`, `shared=37`, `total=10`, `result=147`. The independently recorded seed-2 result is 75, not another interpretation of the same source. New fixture expectations are `3\n1\nnull\n` (`x=2`, `y=3`, `count=1`) and `3\ndivision by zero\n11\n` (`result=3`, `effects=11`). These are bounded-oracle results, not CLI output.
- `node --test web/*.test.cjs`: 39 passed, zero failed/skipped. These are VM/DOM tests, not a real browser session.
- Markdown readback, local link/anchor/command-path checks, production/input/PDF hash preservation and `git diff --check` form the documentation acceptance gate. No server/curl/browser session, full suite, full build, vet or ANTLR generation was run for this refresh.

Contribution audit

Read-only audit commands: `git log d70abf1..633aad1 --format='%h %ad %an %s' --date=short` and `git log --format='%h %ad %an %s' --date=short -- internal/compiscript cmd/compiscript-tac cmd/ide web docs/semestre2/entrega2`.

All ten commits in the first range record author name `jonialen` and date **2026-10-18**: `d592b9e` closures; `3a41adb` objects; `208d709` nominal field correction; `c242e03` lists/foreach; `40d6cf9` codegen report; `2d2fb4e` IDE compilation; `355804c` atomic malformed-loop rejection; `f416fa2` initial report; `9ce55ce` values/effects; `633aad1` guarded dynamic values/handlers. This is nine implementation commits plus the initial documentation commit, not ten newly attested individual contributions. Earlier TAC foundations are recorded on October 17–18 under the same author name. The path history also records earlier IDE work under `DijanU` (`e8c9324`, `cc26b6f`, `3cf44ec` on May 24, and `9ed0d12` on September 6). Those earlier commits are not evidence of a second person's contribution to the recent Entrega 2 units.

A Git author record is not an attestation of the human who performed each task. No teammate percentage, shared authorship, identity equivalence or individual contribution is inferred. Team members must verify their real contribution statements and the assignment's individual-commit requirement before submission. History was not rewritten and no authorship claim was manufactured.

Remaining requirement gaps and presentation checks

Historical gap resolved; no additional concrete normative implementation gap established. Page 1 says “traducen cada construcción de Compiscript”. The initial T8 gap was real and led to S11/T9/T10, not to weakening this requirement. T9 supplies print, remainder, lazy ternary/assignment values, undefined/null storage and null-valued calls; T10 supplies guarded unknown values/untyped formals and synchronous try/catch. Public

core and built-CLI tests now cover the former rejection examples. Independent verification, the five-case field-contract correction/recheck and the source inventory support dispatch for every valid AST construction kind. `BadStmt/BadExpr` remain recovery errors, not missing valid constructs; unary plus/ imports are not grammar constructs.

Syntax/semantic errors still return nil TAC/no codegen. Conservative static global/capture/switch initialization guards and explicit runtime type/null/ undefined/bounds/call faults remain documented contracts. Analyze-valid dynamic-invalid programs can generate guarded TAC and fault in the oracle without becoming new static diagnostics. A coverage inventory is not proof of every program, termination, a full grade or future environmental availability. No further product change is planned without a new concrete requirement or reproduced failure.

Human/evaluation checks still pending: verify real individual contributions, GitHub visibility and availability of the versioned instructions and deliverables; run the battery in the evaluation environment; open the real browser and check `.cps` load/edit/save, Compile/Analyze, located errors, TAC/ symbol views, resets and unchanged YAPar behavior. Confirm CDN availability, layout/accessibility and presentation-day operation. Automated DOM proof does not discharge these steps.

Not an explicit PDF gap: physical byte layout/assembler, a product TAC interpreter and garbage collection are later backend concerns. Abstract cells and retained lexical/frame contracts are documented consistently. Runtime faults in dynamic calls, null/undefined fields and bounds are IR obligations, not promises that every accepted compilation executes without faults. No video or Word export is part of this report. The separately requested four-PDF packet exports existing documentation; it adds no implementation evidence.